

WEB 350: Assignment 9.2 - Governance & AI-Orchestrated Build

STUDENT: Jonathan Cantu

DATE: May 23, 2026

Part I - Governance & Audit Report

1) Structural Instability

- What structural issue caused instability?
- Why did the program appear to work even though behavior changed?
- What lesson did this reveal about async or state-based behavior?

Shared mutable state (`this.lastFormattedCharacter`) and asynchronous callback execution were the structural issues which caused instability. If tested with single/slow writes, the race condition didn't show up. It appears to work when `setTimeout` completes before the next write. Also, no obvious errors are thrown and the code doesn't crash - it just produces incorrect output. Plus the stream still pushes data (`this.push()` still executes successfully).

The critical lessons were:

- Always capture local state for async operations and use closures to preserve data at the time of operation.
- Respect contracts. Don't delay `callback()` in streams - it breaks backpressure. Delaying it causes memory buildup and incorrect flow control.
- Asynchronous operations need isolation. Each async op should own its data.
- Try to include concurrency in your testing. Test with rapid successive operations.

2) Structural Stabilization

- What assumptions were removed?
- What structural improvements made the system predictable?
- How did testing support the repair?

Removed the assumption that the script will always run from the project root, the data directory already exists, any JSON format is acceptable, and missing file returns undefined or throws generic error.

Structural improvements for system predictability:

- Constants for configuration.
- Use `__dirname` instead of `process.cwd()`
- Add `mkdir` with `{ recursive: true }` to ensure directory creation which also prevents `ENOENT` errors.
- Explicit null, 0 parameters in `JSON.stringify` ensure compact, consistent format.
- Explicitly `ENOENT` check returns null for missing files and other errors are properly propagated.
- Encoding specification of "utf8" to prevent buffer vs string confusion and cross-platform compatibility.

The tests helped to isolate the areas where we had issues and provided "visibility" into their nature and eventually covered us for race conditions, error handling, environmental issues, and consistency.

3) Logic Safeguards

Define 3–5 clear rules you would apply when using AI in Node.js development.

Each rule must:

- Be written as a direct statement.
- Include a brief explanation of why it matters.

Rules:

- Never trust AI with async flow control. AI often generates code that looks correct but violates timing contracts.
- AI-Generated file paths require environmental testing. AI defaults to simple patterns like `process.cwd()` that break in GitHub, etc.
- Always capture local state before async operations. AI can create race conditions by storing data in `this.property` or global variables. Closure captures are nearly invisible in code reviews - it takes time to think through execution order during a PR review.
- AI error handling must distinguish error types. AI can treat all errors as equally severe and mask expected conditions.

4) Role of AI vs. Role of the Developer

- What remains the developer's responsibility when using AI?
- What should never be delegated entirely to a generative tool?

Everything is ultimately the developer's responsibility. Nothing changes just because the dev decides to use an AI tool. Regardless of how the solution was arrived upon, the developer is ultimately the one who decides to merge the code and therefore it falls on his/her shoulders to own it.

AI should never have the following responsibilities entirely delegated to it:

- Understanding system contracts and implicit arguments (AI doesn't "experience" production failures)
- Defining failure modes and recovery strategies (AI has no concept of business impact)
- Recognizing environmental and state-based situations. (AI doesn't run code in varied environments)
- Maintaining architectural integrity. (AI has no awareness or context of the project history)
- Writing tests that validate assumptions. (AI writes "happy path" tests which pass trivially. It doesn't know what bugs are expensive in the project domain)
- Security and data privacy. (AI doesn't understand domain specific compliance requirements. And security actually requires adversarial thinking.)
- Production incident response. AI can't read monitoring dashboards or make risk-based decisions.
- Code review and merge decisions. This is dangerous for so many reasons.
- Architecture and design decisions. AI can't balance short-term delivery vs long-term maintainability.
- Performance profiling and optimization. Real performance bottlenecks are often surprising and premature optimization wastes time on wrong problems.
- Dependency selection and upgrades. These definitely need to be reviewed by a developer and also by security tools and specialists.
- Regression analysis and root cause (without manual intervention and review by an actual developer)

Part II - Guided AI-Orchestrated Build

1) Define the Application

In 4-6 sentences, describe:

- The small application you intend to build.
- Why it is appropriate in scope.
- Where asynchronous behavior occurs.
- One intentional failure case the application will handle.

The project must be similar in size and complexity to what you have learned so far in week's 1-6.

Building a small weather app which given the city, state/province, and country will return the current temperature in F, the wind velocity in mph, and the precipitation in inches. There's only a couple simple REST API calls and it will require some async calls because of this so this seems about right for the scope. The application's asynchronous behavior occurs in two main places: the geocoding lookup in `lookupCoordinates()` and the weather lookup in `fetchWeather()`. Both functions use `async/await` with `fetch()`. The intentional failure case the application will handle is malformed location string.

2) Guided AI Generation

The generated application must:

- Be modular (multiple source files).
- Include at least one asynchronous operation.
- Include at least four Jest unit tests.
- Handle at least one intentional failure case.
- Avoid shared global variables.
- Include meaningful inline comments.
- Include a README explaining:
 - What the application does.
 - How to run it.
 - How to run tests.
 - What failure case is handled.

3) Required AI Prompt (Starting Point)

Use this as your foundation:

Generate a small modular Node.js application appropriate for an introductory course. The application must include at least one asynchronous feature, at least four Jest unit tests, one intentional failure case, inline comments explaining logic, and a README explaining usage and testing. Avoid shared global variables. Keep the scope similar to a typical weekly programming assignment.

You may refine wording. You may not expand scope.

Do not forget to stage, commit, and push your work to GitHub (i.e., the AI-generated solution).

Here's my resulting prompt:

Act as a Javascript and Node.js developer with over 10 years of current experience who has a Master's degree in software engineering. Generate a small modular Node.js weather lookup application appropriate for an introductory course. The application should implement as an asynchronous feature fetching weather data (temperature in Fahrenheit, wind velocity, and precipitation in inches) using the open-meteo API using `fetch()`, inline comments explaining logic, and a README explaining usage. Avoid shared global variables. Keep the scope similar to a typical weekly programming assignment. Follow these rules:

- Verify that callbacks, promises, and `async/await` patterns follow the stream and event loop contracts.
- Always capture local state before `async` operations. Confirm that all data used inside is captured in the closure at the time it runs, not read from instance/global state during runtime.
- Error handling must distinguish error types. Avoid generic error handling as this can hide potential bugs by catching and suppressing unexpected errors.

Example request for weather data from open-meteo:

```
https://api.open-meteo.com/v1/forecast?
latitude=43.9956&longitude=-71.1161&current=temperature_2m,wind_speed_10m,
precipitation&temperature_unit=fahrenheit&wind_speed_unit=mph&precipitatio
n_unit=inch
```

Also, the input to the application should be "<city>_<state/province>_<country ISO 3166-1 alpha-2 codes>" (case-insensitive) so to provide the lat-long to open-metro, you will also make use of the open-meteo geocoding API. For example:

```
https://geocoding-api.open-meteo.com/v1/search?name=Boston&count=1
```

An example of a user running this from the command line would be:

```
node app.js Boston_MA_US
```

The application layout should consist of 3 source files: `app.js`, `geocoding.js`, and `weatherService.js`

4) Validation & Explanation

After generating the program, submit a validation write-up (12–15 sentences) that explains:

- What the application does
- Where asynchronous behavior occurs
- How the failure case is handled
- How your safeguards from Part I were enforced
- What you manually reviewed before accepting the generated output

Your explanation is critical. Reasoning matters more than volume of code.

RESPONSE:

What this app does is accepts a city, state/province, and country then outputs a weather report (temperature in F, wind velocity in mph, and precipitation in inches). The format of the input is "<city>_<state/province>_<country code in ISO 3166-1 alpha-2 format>".

The application's asynchronous behavior occurs in two main places: the geocoding lookup in `lookupCoordinates()` and the weather lookup in `fetchWeather()`. Both functions use `async/await` with `fetch()`, which means the network requests are scheduled through the event loop and resumed only after their returned promises settle. Asynchronous behavior also appears in the `run()` function in `app.js`, where the app first awaits geocoding results and then awaits weather results in sequence. The Jest tests validate async behavior by using async test functions, `await expect(blah...).rejects`, and mocked promise-based `fetch` implementations.

The intentional failure case is handled as an expected error-path test rather than as a permanently failing test suite, so it demonstrates failure behavior while still allowing all tests to pass. Specifically, invalid user input such as a malformed location string causes `parseLocationInput()` to throw a `ValidationError`, and that error is asserted in the test.

Failure handling is also differentiated for other cases, including `LocationNotFoundError`, `GeocodingApiError`, `CoordinateValidationError`, and `WeatherApiError`, so the code does not collapse unrelated failures into one generic catch block which is one of the key points driven in this course regarding AI generated code.

Safeguards from Part I were enforced two-fold: By directing the AI to specifically be mindful of the rules (potential pitfalls) and also by implementing carefully architected and reviewed tests. Additionally, inspection of the generated code was an added but critical precaution.

Before accepting the generated output, I manually reviewed the source files for syntax correctness, module exports/imports, argument flow between functions, and whether all known error cases were handled with the correct custom error type. I also manually checked that the tests matched the implementation requirements and that there were at least four tests (9 total between 3 suites) with one intentional failure scenario. I further tested by manually running through several test cases (different city, state/province, and country scenarios) and spot checking results by hitting wunderground.com (for US locations).

And to generate all of this, here is my prompt:

```
-----  
Act as a Javascript and Node.js developer with over 10 years of current  
experience who has a Master's degree in software engineering. Generate a  
small modular Node.js weather lookup application appropriate for an  
introductory course. The application should implement as an asynchronous  
feature fetching weather data (temperature in Fahrenheit, wind velocity,  
and precipitation in inches) using the open-meteo API using fetch(),  
inline comments explaining logic, and a README explaining usage. Avoid  
shared global variables. Keep the scope similar to a typical weekly  
programming assignment. Follow these rules:
```

- Verify that callbacks, promises, and async/await patterns follow the stream and event loop contracts.
- Always capture local state before async operations. Confirm that all data used inside is captured in the closure at the time it runs, not read from instance/global state during runtime.
- Error handling must distinguish error types. Avoid generic error handling as this can hide potential bugs by catching and suppressing unexpected errors.

Example request for weather data from open-meteo:

```
https://api.open-meteo.com/v1/forecast?  
latitude=43.9956&longitude=-71.1161&current=temperature_2m,wind_speed_10m,  
precipitation&temperature_unit=fahrenheit&wind_speed_unit=mph&precipitatio  
n_unit=inch
```

Also, the input to the application should be "<city>_<state/
province>_<country ISO 3166-1 alpha-2 codes>" (case-insensitive) so to
provide the lat-long to open-metro, you will also make use of the open-
meteo geocoding API. For example:

```
https://geocoding-api.open-meteo.com/v1/search?name=Boston&count=1
```

An example of a user running this from the command line would be:
node app.js Boston_MA_US

The application layout should consist of 3 source files: app.js,
geocoding.js, and weatherService.js